Daopin Fu

Senior Java Engineer

~6 years of Java backend engineering · Distributed microservices / smart water / AIoT & GIS / Agent engineering

10w+

Water stations worldwide / GLOBAL WATER STATIONS

- Covers 10w+ water stations worldwide
- Java 21 and Spring Cloud microservices
- Industrial AIoT protocols (UsrCloud/MQTT/Modbus/OP
- PostGIS topology / pruned BFS valve isolation / DMA
- Kafka / Flink realtime streams with optimistic-lock versi
- Hadoop / Spark offline lake and historical backfill
- Elasticsearch hybrid retrieval and affinity scoring
- AgentScope Java 2.0 runtime with sandbox isolation
- Spring AI Alibaba Graph / NL2SQL validation loop
- OA/HR attendance rule engine / performance state machine / SS
- Redis Cluster
- RabbitMQ
- Nacos
- Sentinel

> ENGINEERING STATEMENT // positioning and engineering bar

Java distributed microservices, industrial AIoT and spatial GIS, enterprise OA/HR platforms, reliable realtime/offline data engineering, and production Agent systems. Designs complex business backends, governs high-availability data, and delivers 0-to-1 systems end to end.

★ Strict ownership boundaries · recoverability, consistency, and production observability · end-to-end delivery

TEL: 17322443268 **EMAIL: fudaopin@icloud.com** **BASE: Shenzhen / Guangzhou • full-time**

3D PORTFOLIO SITE: http://cv.bookfree.online/

PROFILE / CAPABILITY MATRIX

Core resume

Senior Java Engineer · 3D museum: http://cv.bookfree.online/ · microservices / AIoT / GIS / Agent

Positioning
Java distributed microservices, industrial AIoT and spatial GIS, enterprise OA/HR platforms, reliable realtime/offline data engineering, and production Agent systems. Designs complex business backends, governs high-availability data, and delivers 0-to-1 systems end to end.

A1 Core competency matrix

Java distributed microservices and multi-tenant architecture
Deep in Java 21 / Spring Boot 3 / Spring Cloud. Owns TransmittableThreadLocal tenant-context propagation, distributed locks, multi-level cache, and cross-service eventual consistency.

Industrial AIoT ingest and thing-model governance
Integrated UsrCloud, MQTT, Modbus TCP/RTU, TCP/UDP meters, FanYi, Maituo, and Gukon OPC/HTTP. Delivered dual-token auth, rate limits, TCP framing, clock checks, and TSL thing-model normalization.

PostGIS topology and DMA leakage algorithms
Uses PostGIS spatial indexes and directed adjacency lists. Designed a pruned BFS to compute minimum valve-isolation sets in milliseconds, plus MNF night-flow leakage analysis.

Agent systems and AI Coding in production
Landed AgentScope Java 2.0 and Spring AI Alibaba StateGraph with persistent Checkpointer, HITL approval, NL2SQL AST checks, and Docker sandbox isolation.

A2 Work experience

Oct 2023 – Present — Litree Water Purification Technology | Senior Java Engineer

Built Litree Smart Water Cloud and the OA/HR platform in parallel: device ledger, multi-protocol ingest, GIS topology, and Agent engineering on the water side; attendance, performance, and master-data sync on the OA side.

- Owned the microservice foundation, device ledger, multi-protocol ingest, GIS/DMA, and data agent for Litree Smart Water Cloud covering 10w+ stations worldwide.
- Independently delivered the OA/HR attendance rule engine, performance state machine, and idempotent master-data sync, then linked org permissions with the water platform.
- Unified station master data, AIoT ingest, GIS topology, and Agent Q&A onto one microservice and permission model so monitoring and the business platform do not keep two account systems.

Oct 2021 – Oct 2023 — Adecco | Java / Big Data Engineer

Worked on Huawei WeLink Xiaowei Search: unified people, groups, and chat retrieval, profile/affinity rules, and 10M-scale realtime/offline ingest pipelines.

- Owned unified retrieval across people, groups, and chats, plus pinyin correction, affinity scoring, and Elasticsearch hybrid-search tuning.
- Built Kafka/Flink realtime and Spark offline dual ingest, killed out-of-order overwrites with external-version optimistic locks, and served as QC/Committer for release reviews.
- Used offline Spark to backfill history and missing links so realtime streams and the lake share one search contract.

Sep 2020 – Oct 2021 — Senge Automation Technology | Java Engineer

Built a smart-water monitoring platform from 0 to 1: field ingest, realtime fan-out, dashboards, alarm-storm control, and containerized delivery.

- Stood up auth, monitoring, reports, and archive from scratch, and connected field ingest to WebSocket dashboards.
- Shipped sliding-window alarm debounce, multi-channel dispatch, idempotent remote control, and Docker Compose multi-env delivery.
- Delivered water-quality/pressure reports, historical archive, and remote control on the same 0-to-1 stack so the plant is operable and replayable.

Technology index
Java 21 · Spring Boot 3 · Spring Cloud · Nacos · OpenFeign · PostgreSQL/PostGIS · Redis · TDengine · MQTT · Modbus · TCP/UDP · OPC/HTTP · AgentScope Java 2.0 · Spring AI Graph · MyBatis · PostgreSQL · Redisson · Quartz · EasyExcel · Java · Spring Boot · HTTP · Modbus TCP/RTU · Thing model

Litree Smart Water Cloud

Litree Water Purification Technology | Oct 2023 – Present

Positioning / system boundary
Unified smart-water platform for 10w+ stations worldwide, connecting station master data, AIoT devices, process monitoring, alarms, GIS networks, analytics, and an in-platform water data agent.

01 Background: context → pain points → goals

Business context	Litree digitally manages 10w+ stations worldwide across project spaces, stations, gateways, IoT devices, process monitoring, alarms, GIS, and analytics. A station's life cycle spans assets, devices, space, and realtime data.
Pain points	Stations, project spaces, gateways, devices, and thing models live in multiple services; identifiers, life cycles, and data contracts are not naturally consistent; Global business needs one identity, tenant, permission, and isolation model while absorbing UsrCloud, MQTT, Modbus, TCP/UDP, OPC, and spatial relations; Monitoring, alarms, network analysis, and agent tools all depend on a trusted data path with explicit contracts, compensation, and ownership boundaries.
Build goals	Connect water objects through unified auth, tenant context, data permissions, and ID mapping; Open the path from station onboarding and device ingest through monitoring, alarms, GIS/DMA, and analytics; Land controllable, recoverable, auditable Agent and AI Coding practice inside the Java stack.

02 End-to-end flow

Project & station → Multi-gateway → IoT ledger → Thing-model → Realtime monitor → GIS topology → Water agent

03 Role and engineering boundary

Role	Owned the device ledger, industrial protocol gateway, GIS topology, and water data agent; joined microservice collaboration, cross-module integration, and production incident work.
Engineering boundary	Owned device ledger, multi-protocol ingest, GIS/DMA, and Agent engineering, covering UsrCloud, MQTT, Modbus TCP/RTU, TCP/UDP meters, FanYi, Maituo, and Gukon OPC/HTTP.

04 Core modules

MODULE 01

Microservice foundation and device ledger

Built a multi-tenant microservice foundation for 10w+ stations to keep multi-source ledgers, multi-gateway topology, and bulk validation consistent.

Role Owned tenant-context propagation, station-gateway-device topology, second-scale bulk ledger import, and cross-service consistency.

Java 21 · Spring Boot 3 · Spring Cloud · Nacos · OpenFeign · MyBatis · PostgreSQL · Redis · Redisson · Quartz · EasyExcel

MODULE 02

Industrial AIoT and GIS networks

Field protocols include UsrCloud, MQTT, Modbus, TCP/UDP meters, FanYi, Maituo, and Gukon OPC/HTTP. Thing models must unify before topology and DMA.

Role Owned UsrCloud/MQTT/Modbus/TCP/UDP/OPC ingest and thing models, plus GIS valve isolation and DMA.

Java · Spring Boot · HTTP · MQTT · Modbus TCP/RTU · TCP/UDP · OPC/HTTP · PostgreSQL/PostGIS · Redis · TDengine · Thing model

MODULE 03

Water data agent and AI Coding

Landed a controllable, recoverable, auditable data agent inside the water microservices, and turned a real water project into an AI Coding workflow.

Role Drove AgentScope Java adaptation and Spring AI agents, including NL2SQL, HITL, sandbox execution, and AI Coding standards.

AgentScope Java 2.0 · Spring AI Graph · NL2SQL · Docker Sandbox · SSE · AGENTS.md

Outcome
Closed the water backend loop from onboarding and ingest through monitoring, alarms, network analysis, and the data agent, supporting 10w+ stations.

TECH MATRIX Java 21 / Spring Boot 3 / Spring Cloud / Nacos / OpenFeign / PostgreSQL/PostGIS / Redis / TDengine / MQTT / Modbus / TCP/UDP / OPC/HTTP / AgentScope Java 2.0 / Spring AI Graph

Litree Smart Water Cloud • Core technical practice

Litree Water Purification Technology | Oct 2023 – Present

MODULE 01

Microservice foundation and device ledger

Role Owned tenant-context propagation, station-gateway-device topology, second-scale bulk ledger import,

Flow Project & station → Multi-gateway → Bulk import → Thing-model → Runtime stats

Boundary Built the foundation on unified auth, end-to-end tenant-context propagation, and ID mapping,

Implementation

- Multi-tenant isolation via TransmittableThreadLocal, Feign interceptors, and MyBatis plugins, with safe cleanup.
- Caffeine + Redis cache and distributed locks to stop stampede, penetration, and avalanches; hot config in L1.
- Compare IoT status by station, drop empty or offline sites, and unify station-gateway-device mapping.
- EasyExcel + JSR-303 row validation for 10k ledgers; failed rows return with locations, no batch abort.
- Idempotent compensation between ledger writes and IoT registration so retries never duplicate devices.
- Generate thing-model attributes from templates; Redisson watchdog locks stop concurrent mock reports and tenant bleed.
- Reconcile station master data with IoT registration on a timer; exceptions enter a compensation queue.

Challenges

- ThreadLocal leaks across pools and schedulers; wrap the life cycle and always clean up.
- Stations have many gateways and stale ledger rows; import failures isolate by row, not roll back the batch.
- Cross-service registration can partially succeed; converge with business-key idempotency and compensation jobs.
- Cache and lock keys must include tenant, or bulk import and registration will cross-write.

Result Opened onboarding and device registration to monitoring, with reconcilable IoT sync.

MODULE 02

Industrial AIoT and GIS networks

Role Owned UsrCloud/MQTT/Modbus/TCP/UDP/OPC ingest and thing models,

Flow Multi-protocol → Device match → Thing-model → PostGIS → Pruned BFS → DMA alert

Boundary Owns full-path ingest and thing-model mapping for UsrCloud, MQTT, Modbus TCP/RTU,

Implementation

- UsrCloud HTTP ingest: Redis dual-token refresh, token-bucket limits, and a bounded pool for paged collection, isolating single-device faults.
- MQTT ingest: product/tenant topic routing, last will, and reconnect; map properties/events into the unified thing model.
- Modbus TCP/RTU and TCP/UDP meters: framing, endianness, CRC, and register high/low mapping.
- FanYi, Maituo, and Gukon OPC/HTTP gateways share a Thing Component interface for config, periodic sync, and fault isolation.
- TSL thing-model normalization: clean points and units; time-window checks stop offline backfill from dirty overwrites.
- PostGIS GIST one-map plus pruned BFS minimum valve sets, with DMA trees and MNF leakage alerts.
- Quarantine unmatched reports so dirty points cannot pollute the thing model or DMA stats.

Challenges

- HTTP clouds, MQTT reports, Modbus registers, and TCP framing differ; the ingest layer must stay decoupled from the unified thing model.
- Bad device clocks or offline backfill can invert time; use windows and latest-state guards.
- Networks have loops and isolated subgraphs; traversals need cycle detection and pruning.
- The same physical device may report over different protocols; merge by thing-model identity.

Result Unified multi-protocol devices on one IoT path with valve isolation and DMA leakage.

MODULE 03

Water data agent and AI Coding

Role Drove AgentScope Java adaptation and Spring AI agents, including NL2SQL, HITL,

Flow User intent → StateGraph → NL2SQL → HITL → Sandbox → Report

Boundary Agents always enforce permission checks, human approval, and sandbox isolation;

Implementation

- Adapted AgentScope Java 2.0 into a HarnessAgent / ReActAgent / Toolkit / Memory / Session runtime.
- Spring AI Alibaba StateGraph orchestrates intent, schema prune, Planner, NL2SQL, Python analysis, and report nodes.
- Wrapped station, device, thing-model, alarm, ticket, GIS, and stats queries as Tool / MCP, clipped by tenant and data permission.
- Persistent Checkpointer supports HITL resume and SSE; AST checks plus Self-Correction constrain NL2SQL.
- Docker pool runs Python analysis against read-only sources with network/FS limits.
- AGENTS.md, Skills, and CI gates close the AI Coding loop from requirement through review and release.
- Tool calls record session, tenant, SQL, and approver; skip HITL and execution is refused.

Challenges

- Long-running agents retry, pause for humans, and drift; they need durable state and SSE session management.
- Models invent cartesian-product or write SQL; block with AST parsing and read-only sources.
- Tools can cross tenants; enforce session, tenant, and data permission at the Tool gate.
- Oversized schemas must be pruned by permission or Planner picks the wrong table or times out.

Result Shipped an approvable, sandboxed water data agent and a durable AI Coding workflow.

Outcome
Closed the water backend loop from onboarding and ingest through monitoring, alarms, network analysis, and the data agent, supporting 10w+ stations.

TECH MATRIX Java 21 / Spring Boot 3 / Spring Cloud / Nacos / OpenFeign / PostgreSQL/PostGIS / Redis / TDengine / MQTT / Modbus / TCP/UDP / OPC/HTTP / AgentScope Java 2.0 / Spring AI Graph

3D PORTFOLIO: http://cv.bookfree.online/

Litree OA / HR System

Litree Water Purification Technology | Oct 2023 – Present

Positioning / system boundary
Internal OA/HR platform covering org/HR, attendance, performance, and SSO, syncing people, orgs, and station master data to the smart-water platform.

01 Background: context → pain points → goals

Business context OA/HR is independent of water monitoring. It owns hire/transfer/leave, flexible shifts, daily attendance close, multi-level performance, and SSO. Water users, orgs, and station permissions depend on this master data, but attendance/performance is not a water-module add-on.

Pain points Attendance rules include overnight shifts, flextime, and leave/overtime offsets; raw punch logs cannot produce reports;
Performance needs plan config, metric rollout, multi-level weights, and version snapshots without losing state;
Org changes must sync idempotently into the water permission domain to avoid duplicate accounts, delayed access, or tenant bleed.

Build goals Deliver the attendance rule engine, performance state machine, and HR life-cycle loop;
Deduplicate in memory by business key and batch-upsert so cross-system master data converges;
Unify auth and org permissions so water accounts take effect as soon as HR changes.

02 End-to-end flow

Org/HR → Shift attendance → Daily close → Performance → Master sync → Water perms

03 Role and engineering boundary

Role Owned the attendance rule engine, performance state machine, org master-data sync, and SSO integration.

Engineering boundary Focused on OA/HR rules and master-data sync; water monitoring, protocol ingest, and Agent capabilities belong to Smart Water Cloud.

04 Implementation and challenges

MODULE 01

Attendance rules and performance

Implementation

- Attendance engine supports flextime and overnight shifts, splits by calendar day, and recognizes holiday make-up days.
- Document-priority offsets combine punches with leave, overtime, and travel into traceable daily-close lines.
- Performance state machine: plan config, metric rollout, self/manager review, weighted scores, and snapshots.
- Shard daily-close batches by person so overnight shifts are not double-counted or dropped at day boundaries.
- Hire/transfer/leave drives account and role changes with an audit trail; shift templates stay decoupled from org calendars.
- Role-templated metrics support weighting, forced distribution, and read-only archives.

Challenges

- Overnight shifts, holiday make-up, and leave priority coexist; daily close must be recomputable and reconcilable.
- Rejected or edited metrics mid-flow need state-machine rollback while keeping historical score versions.
- Month-end close is compute-heavy; avoid long table locks and never score the same person twice concurrently.

Result Attendance close and performance reviews are configurable, recomputable, and auditable.

MODULE 02

Master-data sync and SSO

Implementation

- Quartz paged pulls of OA people, depts, roles, and stations, with in-batch duplicates.
- Dedupe by xtUserId in memory, then PostgreSQL batch upsert with resume-from-failure.
- Clean fields and status; push hire/transfer/leave into water spaces and station permissions.
- OAuth2 SSO; water accounts and permissions take effect as soon as org changes.
- Jobs record batch id, success/fail counts, and conflict samples; failures split by type (PK overwrite, DLQ, soft disable); invalidate water permission cache by org node.

Challenges

- Duplicate pushes and partial failures coexist; write idempotently by business key and never let ON CONFLICT wreck the batch.
- Source data can change mid-page; use watermarks/update windows to catch up.
- Frequent org moves dirty permission cache; expire it and align with SSO sessions.

Result The water platform identifies users and stations through one org permission model; sync is reconcilable and replayable by failure class.

Outcome
Delivered a closed OA/HR platform and kept people, orgs, and station permissions aligned with the water platform.

TECH MATRIX Spring Boot / MyBatis-Plus / PostgreSQL / Redis / Quartz / Spring StateMachine / OAuth2 SSO

Huawei WeLink

Adecco | Oct 2021 – Oct 2023

Positioning / system boundary
Collaboration platform for large enterprises and universities. The open-platform team owned Xiaowei Search: unified retrieval across people, groups, and chats.

01 Background: context → pain points → goals

Business context WeLink Xiaowei Search needs one entry across people, groups, and chats, improved by profiles, personalization, recommendations, correction, and intent. Search consumes live changes and must also absorb history and missing-link backfill.

Pain points People, groups, and chats differ in shape, update rate, and retrieval strategy; Elasticsearch still needs one index contract; Live changes, historical backfill, and missing-link repair coexist; one ingest path cannot cover the full life cycle; Release work must cover data completeness, code quality, and process review.

Build goals Take live changes on Kafka/Flink and history/missing-link repair on Hadoop/Hive/Spark; Sync a unified contract into Elasticsearch for people, group, and chat search; Keep the path maintainable through design docs, QC/Review/Committer, and production delivery.

02 End-to-end flow

Changes & logs → Flink cleanse → Spark reconcile → Optimistic lock → Profile scoring → ES search

03 Role and engineering boundary

Role Led cross-domain retrieval and affinity scoring, owned Flink realtime and Spark offline dual ingest, and served as QC/Committer for release reviews.

Engineering boundary Focused on relevance, lake consistency, and engineering gates; field contracts and query SLAs are agreed with producers.

04 Implementation and challenges

MODULE 01

Unified retrieval and affinity scoring

Implementation

- Unified people/group/chat indexes with IK + pinyin + N-gram hybrid retrieval and Completion Suggester for typeahead and pinyin correction.
- Function Score decay from org-tree distance and interaction frequency to improve first-screen precision.
- search_after cursors instead of from/size deep paging, plus multi-level query cache on hot keys.
- Tuned bulk size, refresh_interval, and routing to control merges and GC.
- Split indexes with aliases so rebuilds stay dark to the query entry.
- Highlight and snippet rules differ by field type so long chats do not drown person-name hits.

Challenges

- Long messages and short person names score differently; tune BM25 weights and decay to avoid bias.
- Heavy writes pressure merges and GC; control bulk, refresh, and replica strategy.
- Org changes drift affinity features; refresh profiles in batches and invalidate score cache.

Result Shipped a unified search entry with correction, completion, and personalized ranking; index rebuilds do not break live queries.

MODULE 02

Realtime / offline dual-path lake

Implementation

- Kafka + Flink cleanse employee, group, and chat index fields near-realtime.
- Hadoop/Spark handle history backfill, scheduled reconcile, and missing-link repair.
- ES version_type= external_gte so offline backfill cannot overwrite newer realtime increments.
- Flink ValueState/BroadcastState holds org dimensions and completes the unified contract before write.
- Dirty data goes to DLQ with alerts; field contracts, partitions, and SLAs confirmed with producers.
- Reconcile jobs emit increment diffs for targeted backfill; persist partitions and watermarks so failed jobs resume without duplicate backfill.

Challenges

- Dual writers invert time unless external-version optimistic locks are used.
- Field contracts diverge; validate schema in-stream before search is polluted.
- Peak traffic needs stable Flink backpressure, checkpoints, and parallelism.

Result Realtime and offline pipes cover the data life cycle; indexes stay reconcilable and resumable.

Outcome
Delivered unified search and dual-path pipelines for 10M-scale enterprise users, where realtime increments and offline backfill coexist without overwriting each other.

TECH MATRIX Java / Spring Boot / Elasticsearch / Kafka / Flink / Spark / Hadoop / Hive / Redis / Huawei Cloud

Senge Smart Water Platform

Senge Automation Technology | Sep 2020 – Oct 2021

Positioning / system boundary
Connected field devices, backend, and clients for plant monitoring, alarms, reports, remote control, and containerized release.

01 Background: context → pain points → goals

Business context The platform was in a 0-to-1 phase: ingest, state handling, realtime monitoring, and alarms extended into water-quality/pressure reports, archive, remote control, and production delivery. Features, realtime paths, and deploy had to land together.

Pain points Field data must travel from ingest and state cache to clients, and faults must enter an alarm operations loop;
No reusable business skeleton existed; auth, monitoring, reports, archive, and audit had to be built together;
Feature work ran in parallel with Docker deploy, release prep, and production incidents.

Build goals Deliver auth, device monitoring/alarms, water-quality/pressure reports, export, and archive;
Build the field-to-dashboard path on Kafka, Redis, and WebSocket; Close a 0-to-1 loop across requirement, design, test, Docker deploy, and production support.

02 End-to-end flow

03 Role and engineering boundary

Role Owned the 0-to-1 platform, spanning the realtime gateway, alarm debounce, auth/reports, and Docker production delivery.

Engineering boundary Led 0-to-1 architecture across realtime channels, business foundation, database tuning, and env-consistent delivery.

04 Implementation and challenges

MODULE 01

Realtime gateway and alarm control

Implementation

- Netty/WebSocket session management, heartbeats, and exponential reconnect with snapshot catch-up.
- Kafka decouples dense time series; Redis caches latest state for dashboards.
- Redis sliding windows aggregate jitter alarms and route SMS, WeChat, and email by severity.
- Remote-control commands carry sequence, signature, and a distributed lock; timeout rolls back so pump/valve commands cannot replay.
- Heartbeats and incremental catch-up restore dashboards after jitter.
- Gateway cluster routes sessions in Redis so reconnects land on the node that holds the snapshot.

Challenges

- Dense bursts become alarm storms; debounce, dedupe, and batch the push.
- Mass reconnect can saturate the gateway; use fast token checks and limits.
- Pump/valve remote control needs atomicity, anti-replay, and strict rollback.

Result Dashboards refresh with low delay, jitter alarms are suppressed, and remote-control commands roll back instead of replaying.

MODULE 02

0-to-1 foundation and container delivery

Implementation

- Spring Boot + MyBatis + MySQL RBAC, unified auth, and audit of sensitive actions.
- Daily/monthly/yearly aggregates and trends for quality, pressure, flow, and power.
- Time-based hot/cold archive: hot data stays queryable, cold data is compacted, primary stays light.
- Composite indexes and summary tables for wide time aggregates.
- Dockerfile and Compose unify dev/test/prod config for one-command deploy; containers inject datasource, Kafka, and alarm-channel config by environment.
- Audit covers login, remote control, export, and permission changes.

Challenges

- 0-to-1 boundaries keep moving; keep layers clear and dependencies tight while still shipping.
- Realtime monitoring and historical reports must use separate channels so long SQL cannot block core IO.
- Scripts, config, and container deps drift across envs; parameterize startup for repeatable delivery.

Result Left behind maintainable auth, reports, archive, audit, and one-command deploy.

Outcome
Delivered a runnable smart-water platform from 0 to 1, connecting monitoring, alarms, reports/archive, and container operations.

TECH MATRIX Java / Spring Boot / Netty / WebSocket / Kafka / Redis / MySQL / Docker